

Comb sort

Comb sort is a relatively simple sorting algorithm originally designed by Włodzimierz Dobosiewicz and Artur Borowy in 1980,^{[1][2]} later rediscovered (and given the name "Combsort") by Stephen Lacey and Richard Box in 1991.^[3] Comb sort improves on bubble sort in the same way that Shellsort improves on insertion sort, in that they both allow elements that start far away from their intended position to move more than one space per swap.

NIST's Dictionary of Algorithms and Data Structures defines comb sort and Shellsort as types of "diminishing increment sort"^[4], but mentions that Don Knuth uses that name as a synonym for Shellsort, while other authors use the name "comb sort" for the entire class.

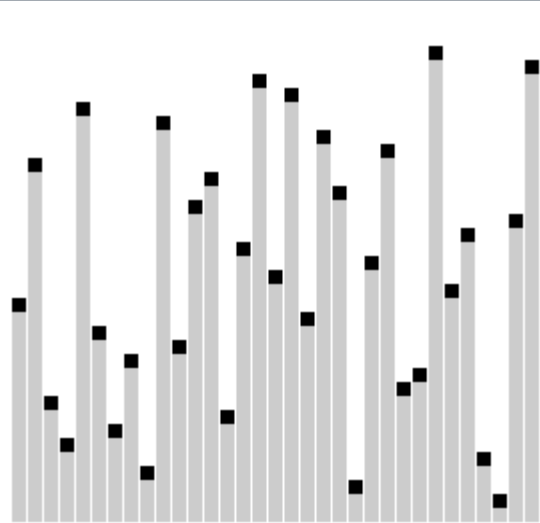
Algorithm

The basic idea is to eliminate *turtles*, small values near the end of the list, since in a bubble sort these slow the sorting down tremendously. *Rabbits*, large values near the beginning of the list, do not pose a problem for bubble sort.

In bubble sort, when any two elements are compared, they always have a *gap* (distance from each other) of 1.^[5] The basic idea of comb sort is that the gap can be much larger than 1. The inner loop of bubble sort, which does the actual *swap*, is modified such that the gap between swapped elements goes down (for each iteration of outer loop) in steps of a "shrink factor" *k*: $[\frac{n}{k}, \frac{n}{k^2}, \frac{n}{k^3}, ..., 1]$.

The gap starts out as the length of the list *n* being sorted divided by the shrink factor *k* (generally 1.3; see below) and one pass of the aforementioned modified bubble sort is applied with that gap. Then the gap is divided by the shrink factor again, the list is sorted with this new gap, and the process repeats until the gap is 1. At this point, comb sort continues using a gap of 1 until the list is fully sorted. The final stage of the sort is thus equivalent to a bubble sort, but by this time most turtles have been dealt with, so a bubble sort will be efficient.

Comb sort



Comb sort with shrink factor $k=1.24733$

Class	<u>Sorting algorithm</u>
Data structure	<u>Array</u>
Worst-case performance	$O(n^2)$ ^[1]
Best-case performance	$\Theta(n \log n)$
Average performance	$\Omega(n^2 / 2^p)$, where <i>p</i> is the number of increments ^[1]
Worst-case space complexity	$O(1)$

The shrink factor has a great effect on the efficiency of comb sort. Dobosiewicz suggested $k = 4/3 = 1.333\dots$, while Lacey and Box suggest 1.3 as an ideal shrink factor after empirical testing on over 200,000 random lists of length approximately 1000. A value too small slows the algorithm down by making unnecessarily many comparisons, whereas a value too large fails to effectively deal with turtles, making it require many passes with a gap of 1.

The pattern of repeated sorting passes with decreasing gaps is similar to Shellsort, but in Shellsort the array is sorted completely each pass before going on to the next-smallest gap. Comb sort's passes do not completely sort the elements. This is the reason that [Shellsort gap sequences](#) have a larger optimal shrink factor of about 2.25.

One additional refinement suggested by Lacey and Box is the "rule of 11": always use a gap size of 11, rounding up gap sizes of 9 or 10 (reached by dividing gaps of 12, 13 or 14 by 1.3) to 11. This eliminates turtles surviving until the final gap-1 pass.

Pseudocode

```
function combsort(array input) is
    gap := input.size // Initialize gap size
    shrink := 1.3 // Set the gap shrink factor
    sorted := false

    loop while sorted = false
        // Update the gap value for a next comb
        gap := floor(gap / shrink)
        if gap ≤ 1 then
            gap := 1
            sorted := true // If there are no swaps this pass, we are done
        else if gap = 9 or gap = 10 then
            gap := 11 // The "rule of 11"
        end if

        // A single "comb" over the input list
        i := 0
        loop while i + gap < input.size // See Shell sort for a similar idea
            if input[i] > input[i+gap] then
                swap(input[i], input[i+gap])
                sorted := false
                // If this assignment never happens within the loop,
                // then there have been no swaps and the list is sorted.
            end if

            i := i + 1
        end loop
    end loop
end function
```

See also

- [Bubble sort](#), a generally slower algorithm, is the basis of comb sort.
- [Cocktail sort](#), or bidirectional bubble sort, is a variation of bubble sort that also addresses the problem of turtles, albeit less effectively.

References

1. Brejová, Bronislava (15 September 2001). "Analyzing variants of Shellsort" (<https://archive.org/download/wikipedia-scholarly-sources-corpus/10.1016%252Fs0016-2361%252802%252900113-8.zip/10.1016%252FS0020-0190%252800%252900223-4.pdf>) (PDF). *Information Processing Letters*. **79** (5): 223–227. doi:10.1016/S0020-0190(00)00223-4 (<https://doi.org/10.1016%2FS0020-0190%2800%2900223-4>).
2. Dobosiewicz, Włodzimierz (29 August 1980). "An efficient variation of bubble sort". *Information Processing Letters*. **11** (1): 5–6. doi:10.1016/0020-0190(80)90022-8 (<https://doi.org/10.1016%2F0020-0190%2880%2990022-8>).
3. Lacey, Stephen; Box, Richard (April 1991). "A Fast, Easy Sort: A novel enhancement makes a bubble sort into one of the fastest sorting routines" (<https://web.archive.org/web/2021092724138/http://cs.clackamas.cc.or.us/molatore/cs260Spr03/combsort.htm>). Hands On. *Byte Magazine*. Vol. 16, no. 4. pp. 315–318, 320. Archived from the original (<http://cs.clackamas.cc.or.us/molatore/cs260Spr03/combsort.htm>) on 2021-09-27. Entire magazine (https://archive.org/details/eu_BYTE-1991-04_OCR) available at archive.org.
4. Black, Paul E.; Kagel, Art S. (7 January 2005). "diminishing increment sort" (<https://xlinux.nist.gov/dads/HTML/diminishingIncSort.html>). *Dictionary of Algorithms and Data Structures*. National Institute of Standards and Technology. Retrieved March 9, 2021.
5. Black, Paul E.; Kagel, Art S. (21 April 2022). "comb sort" (<https://xlinux.nist.gov/dads/HTML/combSort.html>). *Dictionary of Algorithms and Data Structures*. National Institute of Standards and Technology. Retrieved March 11, 2026.

External links

- Calderan, Felipe Vaiano (May 2022). *An analysis of Comb Sort* (https://fvcalderan.github.io/myworks/articles/comb_sort.pdf) (PDF) (Technical report). Federal University of São Paulo (UNIFESP). Retrieved 11 March 2026.

Retrieved from "https://en.wikipedia.org/w/index.php?title=Comb_sort&oldid=1342960648"